

ISB206 Veri Madenciliği

10. Ders: Kümeleme analizi

04.06.2025

Kümeleme analizi

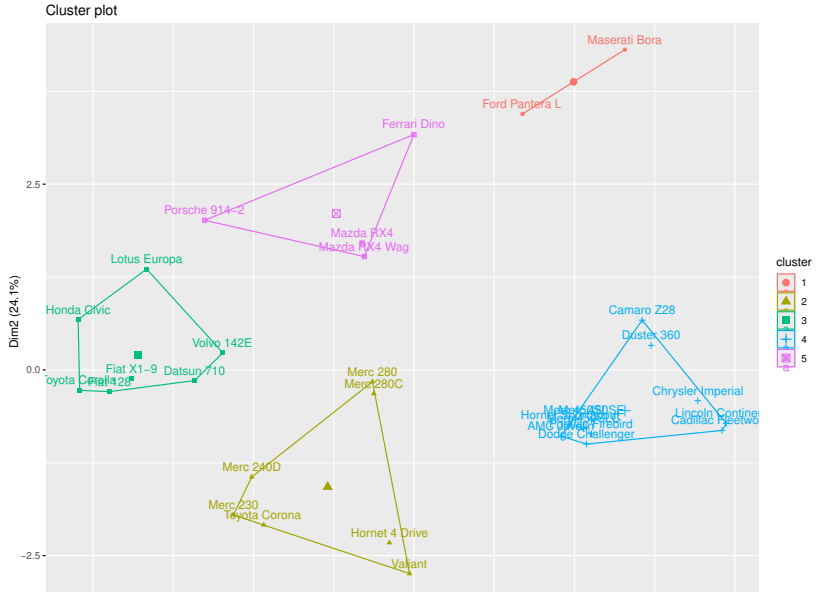
- ▶ Verilerdeki grupları, birbirlerine en yakın/benzeyen gözlemleri ve/veya bir takım desenlerin bulunmasını sağlayan yöntemlerdir.
- ▶ Bu yöntemlerin sınıflama yöntemlerinden farkı, bu yöntemlerde veri önceden bilinen bir sınıf değişkeni içermemektedir. Bu da kümeleme yöntemlerinin gözetimsiz öğrenme yöntemi olduğunu göstermektedir.
- ▶ En bilinen kümeleme algoritmaları K-ortalamalar ve hiyerarşik kümeleme yöntemleridir.
- ▶ Örnek uygulamalar: Müşteri gruplama, anormal işlem belirleme, aşırı büyük veri kümelerini küçültme ya da özetlemek, ...

Küme

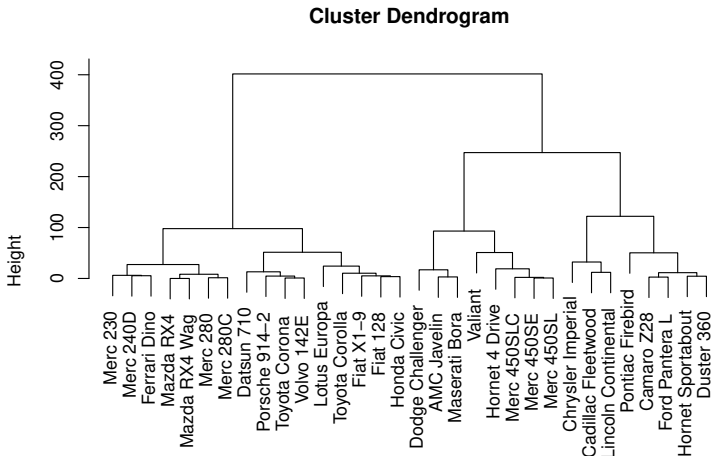
Bu analiz sonucunda elde edilecek kümelerin içerikleri ve yapıları kullanılan yöntemle göre büyük farklılıklar göstermektedir. Örneğin;

- ▶ K-ortalamlar yöntemi kullanıldı ise belirli merkez noktalara en yakın gözlemler bir küme oluşturur.
- ▶ Hiyerarşik kümelemede kümeler ağaç yapısında (dendogram) gruplanır.
- ▶ Modele dayalı kümeleme yönteminde veri karma dağılıma sahip olduğu varsayılır ve her küme bir istatistiksel dağılım gösterir.
- ▶ DBSCAN algoritması veride oluşan desenlere göre kümeler oluşturur. Bu nedenle bu yöntem özellikle coğrafi verilerde kümeleme yapılmasını kolaylaştırmaktadır (Nehir, tarla, park, vb).

MTCARS verisi K-ortalamlar sonucu

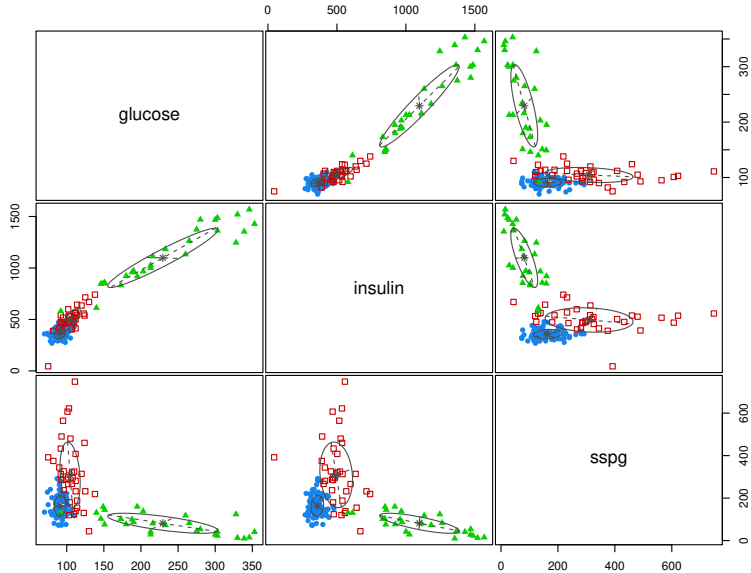


MTCARS verisinin hiyerarşik kümeleme sonucu

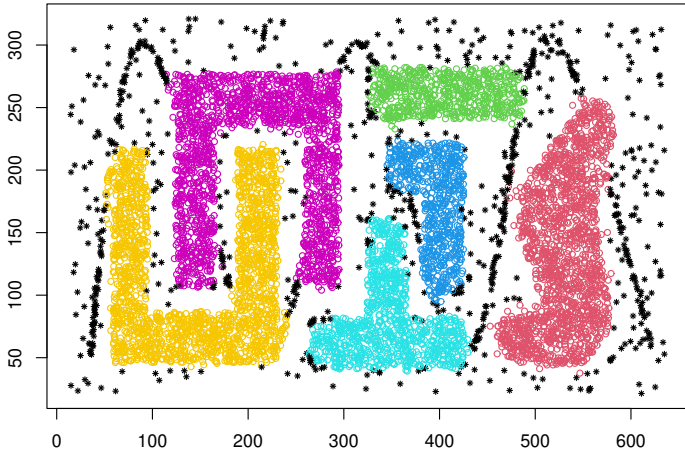


```
dist(mtcars[, 1:3])  
hclust (*, "complete")
```

Modele dayalı kümeleme



DBSCAN



K-ortalamlar

- ▶ Bu derste en temel kümeleme algoritması olan K-ortalamlar incelenecektir.
- ▶ Bu yöntemin kullandığı basit prensipleri anlamanız durumunda, hemen hemen her kümeleme algoritmasını anlamak için gereken bilgiye sahip olursunuz.
- ▶ Bu yöntemde amaç her bir gözlemi daha önceden bizim tarafımızdan belirlenmiş K sayıdaki küme arasında paylaşmaktır.
- ▶ Algoritma temelde iki aşamadan oluşmaktadır. İlk olarak, gözlemleri başlangıçtaki k kümeye atar. Ardından, küme sınırlarını şu anda kümeye düşen örneklerle göre ayarlayarak atamaları günceller.
- ▶ KNN(K en yakın komşuluk) sınıflama yöntemi farklı amaçlara sahip olsalar da birbirlerine benzemektedirler.

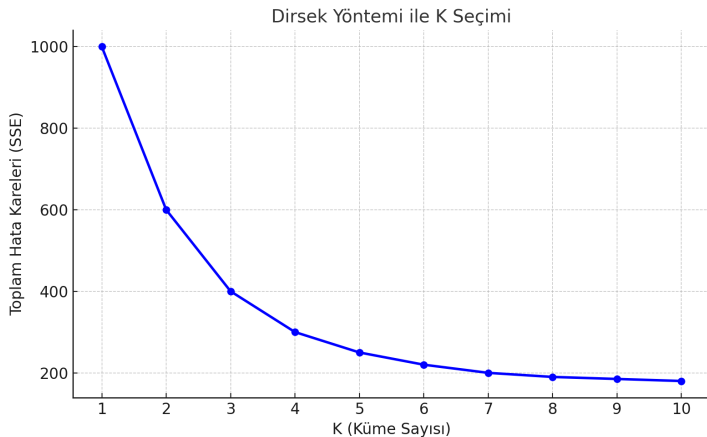
Algoritma

- (1) K elde etmek istediğimiz küme sayısı belirlenir.
- (2) K tane rastgele küme merkez noktası belirlenir.
- (3) Gözlemler en yakın oldukları küme merkezlerinin kümelerine atanır.
- (4) Yeni kümelerin merkez noktası hesaplanır.
- (5) Adım (3) ve (4) artık küme elemanlarında değişiklik olmayıncaya kadar tekrarlanır.

K Değerinin Belirlenmesi

- ▶ K-ortalamalar algoritması, küme sayısı K 'nın önceden belirlenmesini gerektirir.
- ▶ Yanlış bir K seçimi, kümelerin yetersiz veya gereksiz ayrım yapılmasına neden olabilir.
- ▶ En uygun K değeri genellikle kümelerin toplam hata kareleri (SSE) ile analiz edilir.
- ▶ En sık kullanılan yöntemlerden biri **Dirsek Yöntemi**dir.
- ▶ Dirsek yöntemi, farklı K değerleri için SSE değerlerinin çizildiği bir grafikte görsel analiz yapılmasını sağlar.

Dirsek Yöntemi ile K Seçimi



Güçlü ve Zayıf Yönler

Güçlü Yönler

Basit ilkeler kullanır ve istatistiksel olmayan terimlerle açıklanabilir. Oldukça esnektir; basit ayarlamalarla birçok eksikliği giderilebilir.

Gerçek dünya uygulamalarının çoğunda yeterince iyi performans gösterir.

Zayıf Yönler

Daha modern kümeleme algoritmaları kadar detaylı değildir.

Rastlantısal öğeler içerdiği için her zaman en iyi kümeleme çözümünü bulması garanti değildir.

Veride doğal olarak kaç küme olduğu hakkında makul bir tahmin gerektirir.

Küresel olmayan kümeler veya yoğunluğu çok farklı kümeler için ideal değildir.

Uygulama(AVM verisi)

```
> df <- read.csv("AVM.csv", stringsAsFactors = TRUE)  
> str(df)
```

```
df<-df[,4:5] # sadece nümerik veriyi seçelim
```

```
# Optimal küme sayısının belirlenmesi
> skor = rep(0,10)
> for(k in 1:10) skor[k] = sum(kmeans(df, k)$withinss)
> plot(1:10,
      skor,
      type = 'b',
      main = paste('The Elbow Method'),
      xlab = 'K',
      ylab = 'Skor')
```

```
> KM = kmeans(x = df, centers = 5)
> y_KM = KM$cluster

#install.packages("cluster")
> library(cluster)
> clusplot(df, y_KM, lines = 0, shade = TRUE,
           color = TRUE, labels = 2, plotchar = FALSE,
           span = TRUE, main = paste('Müşteri kümeler'),
           xlab = 'Yıllık gelir', ylab = 'Harcama skoru')
```